Prototype Specifications and Recommendations

Tags

Training-Based Multimodal Entity Unlearning with PEFT - Kashish

Remove a specific **multimodal entity concept** from a pretrained system (CLIP-based text-image model such as the text encoder used by Stable Diffusion)

-Text Prompts about the entity no longer produce visual concepts.

-The same pipeline can be scaled to be hardened against adversarial recall.

SETUP

LoRA adapters on:

- CLIP text encoder
- CLIP cross-attention layers for multimodal alignment

LOSS FUNCTIONS

Forgetting Loss- Penalise similarity between generated images and the entity target via CLIP score

Reconstruction/Retention Loss- Maintain normal behavior on retain set

Regularisation- Control magnitude of LoRA weights to avoid destroying unrelated knowledge.

EVALUATION METRICS

Quantitative:

- reduced CLIP similarity between generated/unseen outputs and entity reference
- Stable performance on generic prompts

Qualitative:

- Visual inspection of prompts
- Paraphrase robustness tests

Adversarial Recall Hardening could be Phase 2

CODEBASE-

- GitHub: **AdvUnlearn (PEFT unlearning with adversarial training)**
- GitHub: **few-shot-erasing (BMVC24 code)**

The codebase is for diffusion. But it could be a little harder for us to do diffusion.

COMPUTE POWER

Minimal Prototype (CLIP Text Encoder)- 2-3 GB RAM

CLIP Text Encoder + Cross Attention U-Net- 8-12 GB RAM

Full Diffusion PEFT (CLIP Text Encoder + U-Net Cross-Attention)- 16-24 GB

High-Res / Adversarial Scaling (Full Text Encoder + Cross-Attention + Multi-Step)- 24-40+ GB

Let's Stick to CLIP text encoder only, LoRA rank 2–4, 256×256 resolution, batch size 1.

This will give us real unlearning signals, measurable forgetting, and evaluation of entity removal.

Scaling for adversarial recall

Later we can move to cross-attention layers on cloud GPUs (e.g., 3090, 4090, A100).

we'll just increase LoRA rank, batch size, and resolution.

UCE based Concept Unlearning in Diffusion Models - Rithvika

1. Objective

Perform **Concept Unlearning** from a pretrained text-to-image diffusion model like Stable Diffusion by editing specifically cross-attention weight projections in a **closed-form** manner using the **Unified Concept Editing (UCE)** approach. The goal being

- Remove specific unwanted concepts (an artistic style or unsafe content)
- Preserve quality on unrelated concepts.

References

- *Unified Concept Editing in Diffusion Models (WACV 2024)* – ArXiv / Project page.
- Codebase: https://github.com/rohitgandikota/unified-concept-editing

---

2. Model and Dataset Used

Model (Pretrained)

- **Stable Diffusion v1.4 (or compatible text-to-image diffusion)**
    - U-Net with cross-attention layers linking text tokens to visual generation
    - UCE is then operated on cross-attention parameters in the pretrained diffusion model. No need to train or fine-tune the model

Evaluation Dataset

Evaluation on the unlearning is done on sets of prompts organized into two semantic groups

Concept Type	Example Prompt Set
Forget Concept	Text prompts containing the target concept ("Van Gogh painting of a landscape")
Retain Concepts	Text prompts without the target concept ("Monet style seascape", "Warhol pop art portrait")

---

3. Methodology

Step 1 - Setup

1. **Clone the UCE repository and install dependencies**

```
git clone https://github.com/rohitgandikota/unified-concept-editing.git
cd unified-concept-editing
mkdir models
pip install -r requirements.txt
```

2. **Load a pretrained Stable Diffusion model** from Hugging Face

---

Step 2 - Define Concepts

1. **Concepts to Edit** - `--edit_concepts`
    - One Concept - `["Van Gogh"]` , `["nudity"]`
    - Multiple concepts - `["Van Gogh", "nudity"]`
2. **Concepts to Preserve** - `--preserve_concepts`
    - `["Monet", "Warhol"]`
3. **Target "erased" representation** - `--guided_concepts`
    - How the target unlearned concept should behave when queries - replacement concepts
    - `["art']`

Step 3 - Compute Closed-form Edit

UCE derives a **closed-form weight update** for cross-attention linear layers using

- Concept embeddings (source and target)
- Preservation embeddings

- A matrix inversion solution that wraps prior and new concept alignment constraints

This update is applied directly to model weights without gradient descent.

```
python trainscripts/uce_sd_erase.py \
    --model_id 'CompVis/stable-diffusion-v1-4' \
    --edit_concepts 'Van Gogh; Picasso' \
    --guided_concept 'art' \
    --preserve_concepts 'Monet; Rembrandt; Warhol' \
    --device 'cuda:0' \
    --concept_type 'art' \
    --exp_name '<Expeiment_name>'
```

---

4. Evaluation Metrics

- Evaluation is done by comparing model before and after UCE Edit

A. Concept Removal (Unlearning Performance)

1. **CLIP Similarity Score - Lower the better**
    - CLIP similarity between generated images and the forgotten concept prompt
2. **Prompt success rate**
    - Percentage of generated samples that do not exhibit the forgotten concept when prompted
3. **FID - Lower the better for preservation**
    - Compute FID on a set of images from retained concept prompts

---

5. Hardware Requirements

CPU

- Image Generation - PyTorch + diffusers on CPU will generate images slowly
- UCE - fast (seconds)
- Evaluation - requires image generation and metric computation - slow?
- **Memory:** around 16–32 GB RAM

GPU

- **Modest GPU** (e.g., 6–12 GB VRAM)
    - Significantly faster image generation for evaluation

Cross-lingual Concept Unlearning- Pragya

Primary Objective:

Can unlearning a concept in English transfer to other languages via SAE subspaces?

Hypotheses

The subspace is shared across languages, then removing it in English will also remove it in German and Chinese. SAE features align across languages for shared concepts; removing the English-encoded component of a concept also reduces the model's ability to produce that concept when prompted in other languages.

Model

XLM-RoBERTa-large or mT5-base

Data

Feature identification data: 200 sentences in English and their translations in 2 languages like German and Chinese.
100 sentences with the concept/entity present and 100 without.
SAE training data (double the size of feature identification dataset), Feature identification dataset and evaluation data.

Initially, for an entity like a person's name (like Leonardo da Vinci) or the Eiffel Tower (as that translates differently in different languages).

Method

Train SAEs: On the pooled data of English, German and Chinese, we train the SAE to find features which are language agnostic. This can be done by finding which layers have lowest cosine similarities for pairs like, between (English, German), (English, Chinese) and (German and Chinese).

This way we can train SAE to look for most language agnostic features which point to the entity.

Then on the Feature identification dataset, using SAEs, we must select top the 20 features by mean activation difference (diff-mean when entity is present and not).

Inference Projection

Subspace Construction that consists of collecting the SAE features and find the associated decoder directions that define the subspace. Then we build the projector matrix by doing PCA to retain variance and QR decomposition on the decoder directions, Pc.

The projector **extracts** the component of **h** that points in the directions of the entity, and zeros out everything perpendicular to those directions. So h-Pc@h should remove the entity from the original hidden state.

We can also try a soft removal like,

> h_soft = h - α · (Pc @ h) where value of α will be varied in {0.5, 1.0}, in case there is overlap in the subspace.

For a more permanent change, we can affect the weights

LoRA initialization, from Zhu et al. (2024),

- **B** defines the output subspace (which directions the model can output)
- **A** is just a feature selector (which inputs trigger which directions)

Initialize LoRA B with SAE basis and fine-tune on an English unlearning objective. Then, train to optimize the loss for forgetting the entity.

Evaluation: Test forgetting and paraphrase robustness. Also check utility metrics to see if the unlearning affected other features.

Hardware Req.

Model: 2.4GB
Activations buffer: 1GB
Safety margin: 2GB

Total needed: ~6GB VRAM

→ Works on: RTX 3060 (12GB), RTX 3080 (10-12GB)

References

SAE-guided subspaces

Wang, D., Ma, Q., Shang, Y., Lu, Z., Ning, L., Xu, Z., Wu, H., & He, Z. (2025). ***Interpretable Safety Alignment via SAE-Constructed Low-Rank Subspace Adaptation*****. arXiv:2512.23260.**
This paper introduces a method for constructing an interpretable low-rank subspace for fine-tuning large language models by leveraging Sparse Autoencoders (SAEs). Instead of letting LoRA implicitly learn a task-specific subspace, the authors first identify SAE features whose activations differ between aligned and unaligned behaviors. This uses the method for safety alignment.

SSPU

Wang, X., Li, Y., & Zhang, H. (2025). ***Model Unlearning via Sparse Autoencoder Subspace Guided Projections*****. EMNLP 2025.**
The SSPU paper introduces a subspace-projection framework for model unlearning that leverages Sparse Autoencoders (SAEs) to isolate concept-specific directions in a model's activation space. Instead of modifying model weights directly, the method identifies SAE features that strongly encode the target concept, extracts their decoder directions, and constructs an orthonormal forgetting subspace using PCA and QR decomposition. During unlearning, model activations are projected onto the orthogonal complement of this subspace, effectively removing the concept-related components while preserving unrelated knowledge.